Lec 11 Large-Scale Distributed Training. 大规模分布式训练.

在今日的实践中，这是几乎所有模型的训练方式.

Running Example: Llama 3 - 405B. (示范性与艺术性).

(过去几家人工智能公司并未开源其模型细节，而Llama3正是最开放的模型).

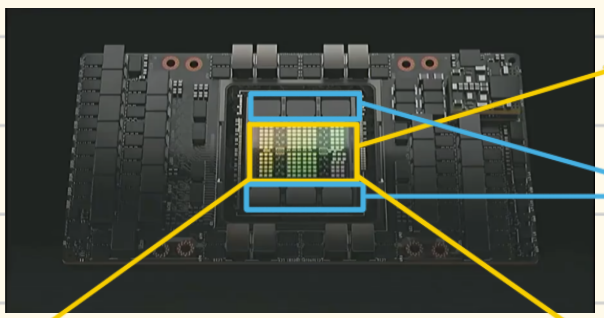
Today: GPUs and How to Train On Them.

- GPU Hardware
- How to train on GPUs.

GPU: NVIDIA H100.

GPU: Graphics Processing Unit. 图形处理单元，最早是为计算机图形学开发的协处理器.

现在慢慢发展成了通用并行计算的处理器.

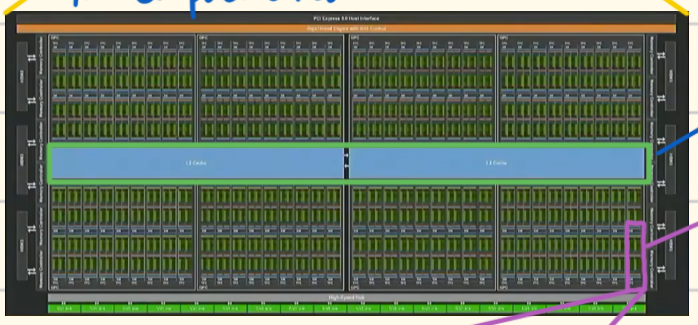


→ Compute Cores.

高带宽内存. → 通过总线访问，连接(~3TB/s).

80GB HBM Memory.

H100 Compute Cores:



50MB L2 Cache (更快的访问).

流式多处理器.

132 Stream Multiprocessors (SMs) 独立的平行内核.

(理论有144个，但由于缺陷会少，至少有132个).

H100 Streaming Multiprocessor.

(132中的1个).



256kB L1 cache, 256kB registers.

128 FP32 Cores

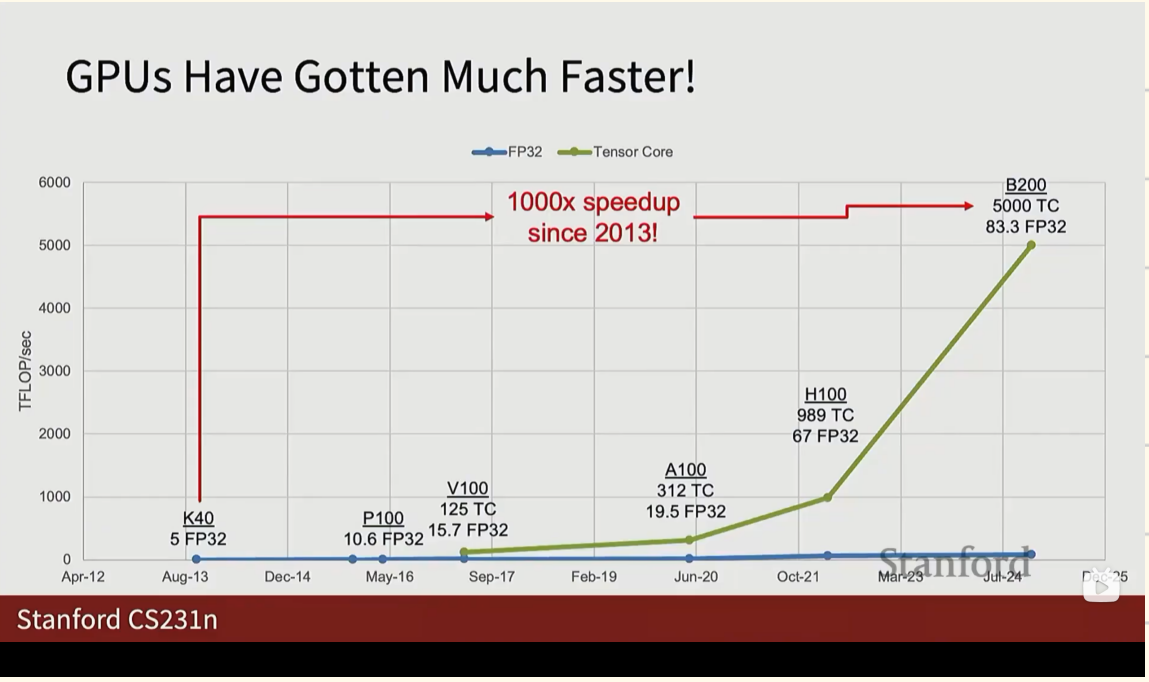
浮点运算的基本单位，可在1时钟周期计算 ax+b (标量). 256 FLOP/cycle·SM

4 Tensor Cores. (矩阵核心).

可完成单块矩阵乘法. Ax+B. 1时钟周期 409 FLOP/cycle·SM

张量核是SM计算的主要吞吐量.

Mixed percision: 16 bit/32 bit (以16入，以32出).



在过去10年中，GPU设备的计算能力提升了1000倍，这也是AI或DL取得进展的重要原因..

通过并行GPU，导致计算能力更增强.

H100 GPU：3352 GB/s 内存带宽.

Server（服务器）：8×GPU：900 GB/s between GPUs.

Rack = 2 Server = 16 GPUs：
（机架）.

GPU Pod = 192 Racks = 3072 GPUs. 50 GB/sec between GPUs.

Cluster = 8 Pods = 24576 GPUs. < 50 GB/sec.

（Llama3 论文参考）.

- 24576 GPUs
- 1.75 PB of GPU Memory
- 415M FP32 cores
- 13M Tensor cores
- 24.3 EFLOPs/s = 24.3×10^{18}

- Google：Tensor Processing Units（TPUs）张量处理单元.
- AMD：MI325X　　AWS：Trainium2.

拆分你的计算：训练巨型神经网络.

对Transformer来说：L layers，operates on tensors of shape（Batch, Sequence, Dim）

⇒ 依此，可在4个轴向上拆分/并行：

① Pipeline Parallelism（PP）.
（在L上拆分）

② Data Parallelism（DP）
（在Batch上拆分）

③ Context Parallelism（CP）
（在Sequence上拆分）.

④ Tensor Parallelism（TP）
（在Dim上拆分）.

} 拆分计算的不同方式.

DP: Data Parallelism

Recall: 训练网络通常在小批量上计算Loss.

对batch上的每个元素，Loss计算与梯度计算是独立的

idea: 对MN大小的Minibatch：在M个GPU上，每个GPU计算N个

梯度是线性的：$Loss = \frac{1}{MN}\sum_{i=1}^{M}\sum_{j=1}^{N} loss(x_{ij}, w)$ cLoss是整个Minibatch中每个人损失的平均值

$$\Downarrow$$

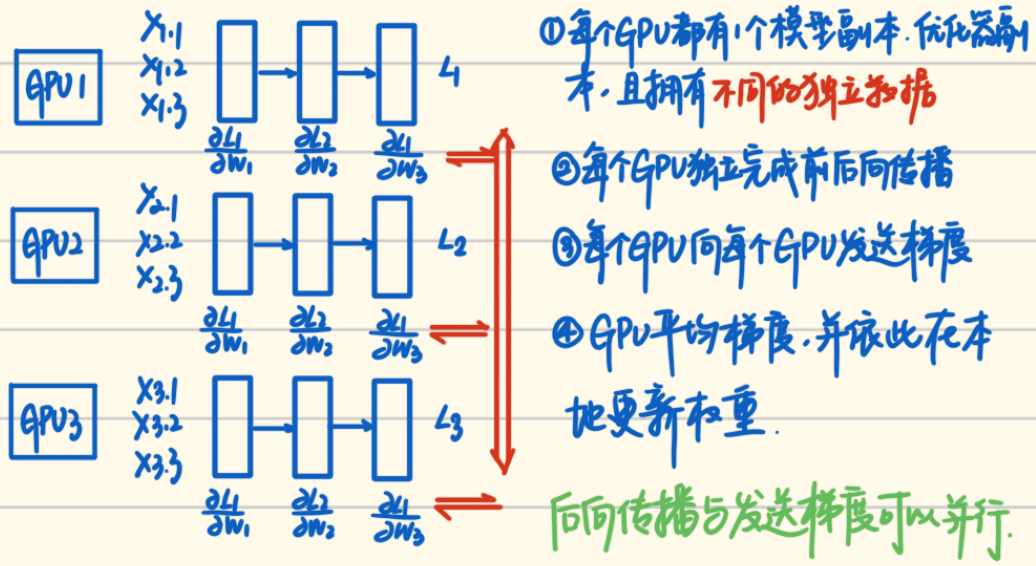
$$\frac{\partial Loss}{\partial w} = \frac{1}{MN}\sum_{i=1}^{M}\sum_{j=1}^{N}\frac{\partial}{\partial w} loss(x_{ij}, w)$$

通过Loss的线性性，在梯度上拆分.

$$= \frac{1}{M}\sum_{i=1}^{M}\left(\frac{1}{N}\sum_{j=1}^{N}\frac{\partial}{\partial w} l(x_{ij}, w)\right)$$

紫色在单个GPU上完成计算 再最后求平均值.

完全等价，而非近似.



① 每个GPU都有1个模型副本，优化器副本，且拥有不同的独立数据

② 每个GPU独立完成前后向传播

③ 每个GPU向每个GPU发送梯度

④ GPU平均梯度，并依此在本地更新权重.

后向传播与发送梯度可以并行.

瓶颈：Model Size

每个权重有4个值(w, grad w, Adam β_1, β_2).

以float(16)需 8Byte. 即：

1B参数需8GB内存空间

解决方式：拆分模型权重.

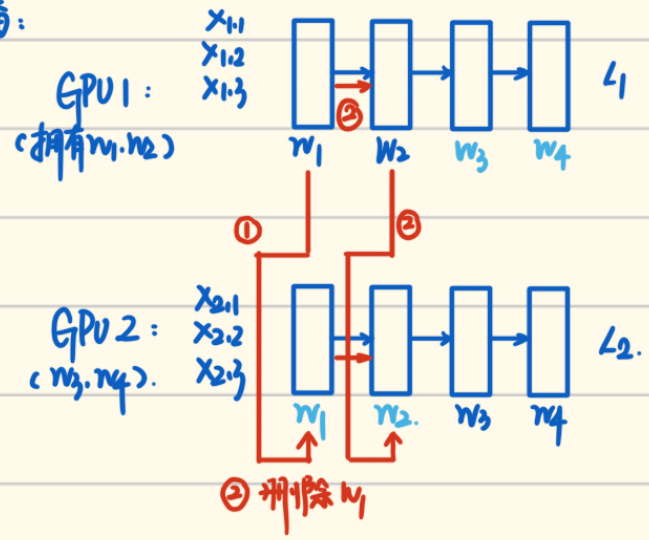
Fully Sharded Data Parallelism (FSDP). 完全分片数据并行.

将模型参数拆分到不同GPU上. ⟶ 每一个GPU负责有限层的权重矩阵存储.

同时也负责管理权重的全局梯度和优化器状态.

以2个GPU为例：

前向传播：



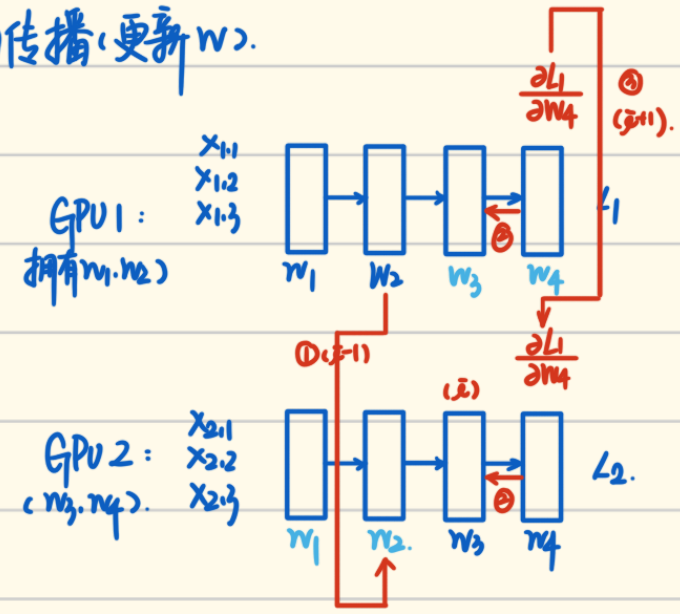
① 引入额外的交流：将"即将向前传播(i层)"的矩阵广播. 其余GPU接受副本

② 所有GPU执行向前传播，并在完成后删除权重矩阵的副本.

在进行i层的③时，同时进行i+1层的①(预取).

直到进行到最后一层，计算出Loss后开始反向传播.

反向传播(更新W).



(W4在正向的最后一步，故所有GPU都可不删除副本).

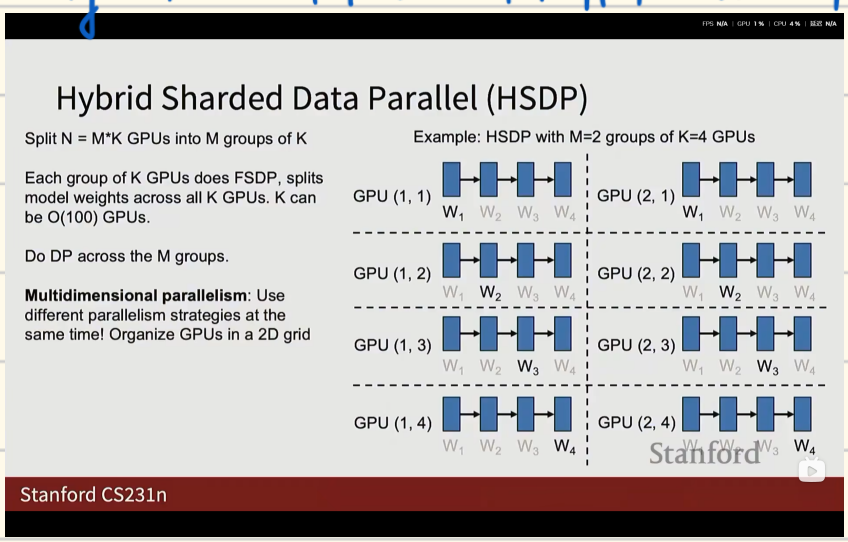
①权重所有者广播第i层的权重，其余GPU得到并保存.

②各个GPU计算本地向后传播 $\frac{\partial L_n}{\partial W_i}$, $\frac{\partial L_n}{\partial X_i}$，删除Wi副本.

③各个GPU广播 $\frac{\partial L_n}{\partial W_i}$，权重所有者收集并求和，存储结果.

parallel：进行i层②同时，进行i-1层的广播(获取)，并进行i+1层的③(求和).

Hybrid Sharded Data Parallel：混合分片数据并行，HSDP.



将GPU分成单组约100个的组，组内进行FSDP，组外普通DP(即仅在完成前后向传播时广播以更新权重).

FSDP前后向加和要相互交流3x的权重数，而普通DP仅需1x权重矩阵交流.

多层次的思想，适用于不同远近的网络拓扑结构.

Problem：模型"激活值"：即神经网络前向传播时，每一层都需保留的"中间结果"(cache)，在反向传播计算完后才可释放。且激活值与batch数据有关，GPU必须完整保留

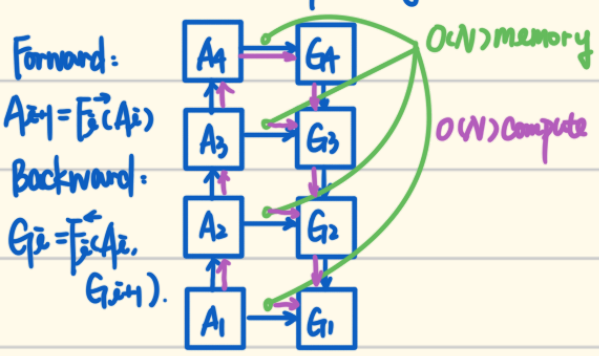
在layers足够大时，激活值将占用所有显存空间。

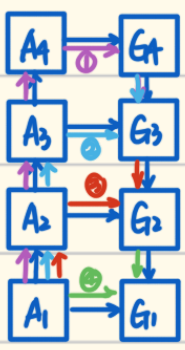
以Llama3-405B来说：有126 layers，D=16384，sequence=4096，则transformer的所有FFN隐藏维激活值就会占据 $2\times(4\times16384\times4096)\times126$ bytes (以FP16精度，隐藏层4D维度).

Solution：Activation Checkpointing 激活值重计算.

在前向传播不保存激活值，在反向传播需要时，重新计算一遍前向传播.

Activation Checkpointing.



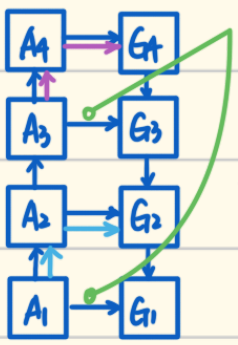


O(1) Memory

$O(N^2)$ Compute.

以空间换时间的计算策略.

idea：每C层记录一次Activation



$O(N/c)$ memory.

$O(N/c \cdot c^2)$ Compute

$=O(Nc)$

$c=\sqrt{n}$ → $O(N)$ memory, $O(NN/N)$ Compute

平衡空间.时间.

How to Train on Lots of GPUs.

HSDP + Activation checkpointing can take you a long way!

(混合数据并行 + 激活值重计算足以支撑很多模型的训练.)

根据参数量:

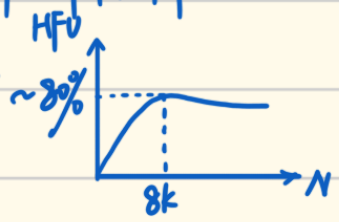
1. 在~1B参数时. 可使用~128 GPUs进行数据并行.
2. 根据每个GPU的内存量设置每个GPU的batch size.
3. 如果model的大小 >1B参数, 则考虑FSDP.
4. 添加Activation Checkpointing以适应更大的batches/GPU.
5. 若GPU数 >256, FSDP的开销就很大了, 考虑HSDP. } 百亿级. 近千GPU.
6. 超过1000 GPUs, 超过500亿参数模型, 序列超过16000, 考虑更多并行策略(CP, PP, TP).

Problem: { 全局批量. 本地批量大小,
HSDP维度, FSDP维度.
Activation Checkpointing 大小 } 参数太多, 应该怎么调整?

Solution: 指标 Model Flops Utilization (MFU) 模型浮点运算利用度. 衡量GPU性能.

Hardware FLOPs Utilization (HFU) 硬件浮点运算利用率.

HFU = 实际每秒计算次数 / 理论每秒计算次数



Problem: HFU不能衡量除矩阵计算外其他GPU操作

(例如激活值重计算、数据增强等).

Model FLOPs Utilization (MFU).

idea: GPU中理论最大FLOPs计算中有多少比例被用于模型计算.

1. 根据模型的特征(架构.参数...), 计算出 forward + backward 的理论浮点运算次数.

   $FLOP_{theoretical}$ = total number of matrix multiply FLOPs in the forward + backward pass.

2. 得到当前设备的理论最大吞吐 $FLOP/sec_{theoretical}$

3. 计算理论耗时 t

   $t_{theoretical} = FLOP_{ther} / FLOP/sec_{ther}$.

4. 测量实际耗时 t_{actual}.

5. $MFU = t_{theo} / t_{actual}$

MFU: >30%: good

>40%: excellent.

在Llama3-405B上, MFU得到约40%的结果.

最近的设备可能会更差(GPU通信增速慢于处理增速).

Context Parallelism: 在序列上分割.

核心难点: Attention 的核心矩阵. 难以分割.

- 环形注意力
- Ulysses 注意力: 增加多头注意力的数目, 并行多头.

Pipeline Parallelism (PP): 在层上拆分.

核心难点: 数据依赖性. 必须等待而产生Bubble

解决方式: 多批数据并行(但有上限, 存储的激活值随批数增多而变大).

Tensor Parallelism (TP): 在权重矩阵计算上拆分. (分块矩阵).

问题: 在计算后需收集分块的矩阵.

实践中, 我们使用**所有的并行机制!**